



**Universidade Federal
do Espírito Santo**

Técnicas de Busca e Ordenação
Trabalho Prático 1

[João Arthur Andreatta Subtil](#)
[Leonardo Cheregati de Oliveira Roxo](#)
[Matteo Chiste Carvalho Trento](#)

SUMÁRIO

1. INTRODUÇÃO.....	2
2. METODOLOGIA.....	4
2.1. Tipos Abstratos de Dados (TADs) Principais.....	4
2.2. Estruturas de Dados Principais.....	4
2.3. Decisões de Otimização e Algoritmos.....	4
3. ANÁLISE DE COMPLEXIDADE.....	6
3.1. Leitura dos Dados e Geração dos Vértices.....	6
3.2. Cálculo das Distâncias (Geração das Arestas).....	6
3.3. Construção do Min Heap Binário.....	6
3.4. Obtenção da MST (Kruskal com Union-Find).....	6
3.5. Identificação dos Grupos e Escrita do Arquivo.....	7
3.6. Complexidade Total do Algoritmo.....	7
4. ANÁLISE EMPÍRICA.....	8
REFERÊNCIAS.....	10

1. INTRODUÇÃO

Este relatório apresenta o desenvolvimento e os resultados do algoritmo de Agrupamento de Espaçamento Máximo, referente ao Trabalho Prático T1 da disciplina de Técnicas de Busca e Ordenação.

De forma simples, o agrupamento (ou *clustering*) tem como objetivo encontrar "grupos naturais" em uma base de dados. A lógica principal é organizar os elementos de maneira intuitiva: juntar objetos que são parecidos (similares) no mesmo grupo e garantir que objetos diferentes fiquem em grupos separados. Neste trabalho, a diferença entre os pontos (representados no espaço R^m) é medida pela distância euclidiana.

As entradas contam com n pontos, cada um de dimensão m . O desafio central é dividir esses pontos em uma quantidade k de grupos de forma que o espaço (a distância) entre grupos diferentes seja o maior possível. Para resolver o problema, utilizamos a construção de uma Árvore Geradora Mínima (MST) através do Algoritmo de Kruskal. A implementação foi desenvolvida na linguagem C, priorizando a clareza e a eficiência estrutural.

A principal dificuldade encontrada na implementação do trabalho residiu na otimização da criação e da ordenação das arestas. Contudo, os resultados obtidos comprovam que o método de não adicionar as $k-1$ maiores arestas à MST é eficaz para formar os agrupamentos finais esperados em menos tempo.

2. METODOLOGIA

2.1. Tipos Abstratos de Dados (TADs) Principais

Para modularizar o código e facilitar a manipulação das informações, o sistema foi dividido em três TADs principais:

- **Vértice:** Representa um ponto no espaço R^m .
 - **id:** Cadeia de caracteres (string) que armazena o identificador único do ponto.
 - **coord:** Vetor de double que guarda as coordenadas espaciais do ponto.
 - **qtd_coord:** Inteiro que armazena a dimensão **m** do ponto.
 - **pai:** Ponteiro que indica o nó ancestral no conjunto *Union-Find*.
 - **tam_arvore:** O tamanho da componente conexa no *Union-Find*.
 - **primeiro:** Ponteiro que aponta sempre para o vértice com o menor **id** lexicográfico dentro daquele grupo, facilitando a ordenação da saída.
- **Aresta:** Modela a conexão entre dois pontos.
 - **v1** e **v2:** Índices para os vértices de origem e destino da aresta, que estão armazenados em um vetor externo (presente no **Espaço**, detalhado a seguir).
 - **tam:** Variável double que armazena o "peso" da aresta, ou seja, a distância entre **v1** e **v2**.
- **Espaço:** Atua como o gerenciador central do ambiente.
 - **vertices:** Vetor de ponteiros para as instâncias de **Vértice**.
 - **arestas:** Vetor contendo todas as instâncias de **Aresta** geradas.
 - **qtd_arestas:** Inteiro que controla o número total de arestas.

2.2. Estruturas de Dados Principais

- **Vetores (Arrays Dinâmicos):** Foram amplamente utilizados devido ao acesso em tempo constante $O(1)$ e à localidade de referência, que otimiza o uso do cache do processador. O vetor de vértices, por exemplo, é alocado dinamicamente e redimensionado em blocos (usando a constante **TAM_VET**) via `realloc`, evitando realocações a cada nova linha lida do arquivo.
- **Árvores (Florestas de Conjuntos Disjuntos):** A base lógica para identificar os grupos reside na modelagem em árvores por meio da estrutura *Union-Find*. Inicialmente, cada vértice é uma árvore isolada (raiz de si mesmo). À medida que o algoritmo avança, essas árvores são conectadas, formando uma floresta onde cada árvore final representa um dos **k** grupos.

2.3. Decisões de Otimização e Algoritmos

- **Cálculo da Distância Euclidiana sem Raiz Quadrada:** Na função de cálculo de distância (`calculaEuclides`), optou-se por somar o quadrado das

diferenças entre as coordenadas, mas omitiu-se a extração da raiz quadrada. Como o objetivo do trabalho é apenas comparar as distâncias para ordená-las, a relação de ordem se mantém perfeitamente (se $a > b$, então $a^2 > b^2$ para valores positivos de ambos), economizando processamento matemático custoso.

- **Otimizações do Union-Find:** Foram implementadas as duas heurísticas clássicas para garantir eficiência quase linear:
 - *Union by Size:* Na função **une_vertices**, a árvore com menor número de nós é sempre anexada como subárvore da raiz da árvore maior, mantendo a profundidade reduzida.
 - *Path Compression (Compressão de Caminho):* Durante a busca pelo representante do grupo (**encontra_raiz**), os nós visitados têm seus ponteiros **pai** atualizados diretamente para a raiz principal, achatando a árvore para futuras consultas.
- **Algoritmo de Kruskal Adaptado (Early Stopping):** O laço principal do Kruskal não percorre e tenta unir vértices até esgotar as arestas. O algoritmo é interrompido antecipadamente assim que $n - k$ arestas válidas são adicionadas, pois, por definição teórica, uma floresta com n vértices e $n - k$ arestas possui exatamente k componentes conexas. Ademais, a seleção da menor aresta não é feita sequencialmente no vetor, mas sim retirando-se o primeiro elemento da estrutura Min Heap Binário, citada a seguir. A cada iteração, o primeiro elemento é retirado, sendo substituído pelo último. Então, a função **heap_down** reorganiza o Heap.
- **Estratégias de Ordenação Híbridas:** No lugar da ordenação inicial do массивo vetor de arestas (etapa de maior custo do código), o algoritmo foi modificado para usar uma fila de prioridade implementada com o **Min Heap Binário**. Testes empíricos demonstraram que esse algoritmo lidou de forma mais previsível e rápida com esse volume de dados específico, em comparação a outras abordagens. Por outro lado, para a ordenação final do vetor de vértices (para adequar a formatação de saída por ordem lexicográfica), utilizou-se a função nativa **qsort** (*Quick Sort*), que se provou suficiente para essa tarefa secundária.

3. ANÁLISE DE COMPLEXIDADE

Para estabelecer a análise assintótica do algoritmo desenvolvido, definiremos a seguinte variável baseada nos parâmetros de entrada do problema:

E: número total de arestas geradas no grafo. Como o problema modela um grafo completo (onde todos os pontos se conectam entre si), o número exato de arestas é dado por $E = \frac{n(n-1)}{2}$, o que nos permite aproximar o crescimento para $E \sim \frac{n^2}{2}$.

Abaixo, detalharemos o custo computacional no pior caso para cada uma das etapas principais da execução.

3.1. Leitura dos Dados e Geração dos Vértices

A leitura iterativa percorre as **n** linhas do arquivo de entrada. Para cada linha, são extraídos o identificador e as **m** coordenadas correspondentes. A alocação dinâmica no vetor de vértices ocorre em blocos (via *realloc*), resultando em um tempo amortizado constante.

- **Complexidade: $O(n * m)$**

3.2. Cálculo das Distâncias (Geração das Arestas)

A construção do grafo exige o cálculo da distância (sem a extração da raiz quadrada, como otimização) entre todos os pares de pontos possíveis. Para cada uma das **E** arestas geradas, o algoritmo realiza um laço iterando sobre as **m** dimensões. O custo de operações é proporcional a $E * m$. Substituindo a aproximação de **E**, temos $\frac{n^2}{2} * m$.

- **Complexidade: $O(n^2 * m)$**

3.3. Construção do Min Heap Binário

A ordenação do vetor contendo todas as arestas foi implementada utilizando a estrutura de dados Min Heap Binário, cuja complexidade clássica de construção é $O(E)$. Substituindo **E** pela sua aproximação em função do número de vértices, obtemos $O(n^2)$. Além disso, a constante pode ser descartada da notação.

- **Complexidade: $O(n^2)$**

3.4. Obtenção da MST (Kruskal com Union-Find)

No limite superior (pior caso), o algoritmo de Kruskal iteraria sobre todas as **E** arestas. Graças às heurísticas de *Union by Size* e *Path Compression* aplicadas à estrutura *Union-Find*, o custo das operações de busca e união é praticamente constante: $\sim O(1)$. Logo, o custo é dominado pelo número de arestas visitadas. Devido ao critério de parada antecipada (*early stopping*), o laço encerra na iteração

em que $n - k$ junções válidas são realizadas, reduzindo drasticamente o tempo na prática. Ainda assim, para a notação de limite superior: teríamos $O(E \lg E) \sim O(n^2 \lg n^2)$. Aplicando a propriedade logarítmica, essa complexidade é igual a $O(2 * n^2 \lg n)$. Além disso, podemos eliminar a constante para obter a complexidade final:

- **Complexidade: $O(E \lg E) \sim O(n^2 \lg n)$**

3.5. Identificação dos Grupos e Escrita do Arquivo

Após a finalização do agrupamento, o vetor de vértices é ordenado lexicograficamente por meio do *Quick Sort* nativo (qsort), o que custa $O(n \log n)$. A varredura final para impressão e escrita no arquivo de saída custa $O(n)$. Como a ordenação domina o crescimento assintótico desta fase, o termo linear é absorvido.

- **Complexidade: $O(n \log n)$**

3.6. Complexidade Total do Algoritmo

Somando as etapas principais abordadas, o limite de processamento do programa é ditado de forma esmagadora pelas etapas de cálculo das distâncias do massivo vetor de arestas. Assim, a complexidade assintótica global do sistema é estabelecida pela soma dos termos de maior grau de crescimento:

- **Complexidade Final: $O(n^2 * m + n^2 \log n)$**

4. ANÁLISE EMPÍRICA

Para validar a análise de complexidade teórica da Seção 3, esta etapa apresenta os resultados práticos da execução do programa. O objetivo é medir o tempo de processamento de cada fase do algoritmo para confirmar se os resultados práticos correspondem às expectativas matemáticas.

Antes de apresentar os dados, vale ressaltar que, como a medida do tempo total não considera o tempo de cálculo e impressão do tempo em si, esse pode ser ligeiramente menor que o tempo real da execução do programa. Por essa razão, é possível que o percentual que representa o total do algoritmo na **Tabela 2** seja um pouco menor que 100%.

Os dados extraídos dos testes de performance estão organizados nas seguintes tabelas:

- **Tabela 1:** Tempos de execução absolutos de cada etapa do algoritmo para diferentes instâncias de teste, em segundos.

Etapa	1	2	3	4	5	6	7	8
Leitura dos Dados e Geração dos vértices	0.000 0	0.000 1	0.000 5	0.002 7	0.009 9	0.027 5	0.105 9	0.4142
Cálculo das Distâncias (Geração das Arestas)	0.000 0	0.000 1	0.009 3	0.080 9	0.471 6	3.127 5	23.98 80	185.03 34
Construção do Min Heap Binário	0.000 0	0.000 1	0.011 5	0.067 5	0.269 3	1.075 9	4.286 8	17.497 1
Obtenção da MST (Kruskal com Union-Find)	0.000 1	0.000 2	0.022 8	0.112 9	0.322 6	0.127 9	0.543 2	2.3005
Identificação dos Grupos e Escrita do Arquivo	0.000 1	0.000 1	0.000 2	0.000 2	0.000 4	0.000 6	0.001 4	0.0038
Desalocação de memória	0.000 0	0.000 0	0.000 5	0.003 3	0.011 1	0.041 1	0.180 4	0.7346
Total do Algoritmo	0.000 3	0.000 6	0.044 9	0.267 6	1.084 9	4.400 6	29.10 58	205.98 37

- **Tabela 2:** Tempos de execução relativos de cada etapa do algoritmo para diferentes instâncias de teste, representados como porcentagem do tempo total daquele caso.

Etapa	1	2	3	4	5	6	7	8
Leitura dos Dados e Geração dos vértices	0.00 %	16.67 %	1.11%	1.01%	0.91%	0.62%	0.36%	0.20%
Cálculo das Distâncias (Geração das Arestas)	0.00 %	16.67 %	20.71 %	30.23 %	43.47 %	71.07 %	82.42 %	89.83 %
Construção do Min Heap Binário	0.00 %	16.67 %	25.61 %	25.22 %	24.82 %	24.45 %	14.73 %	8.49%
Obtenção da MST (Kruskal com Union-Find)	33.33 %	33.33 %	50.78 %	42.19 %	29.74 %	2.91%	1.87%	1.12%
Identificação dos Grupos e Escrita do Arquivo	33.33 %	16.67 %	0.45%	0.07%	0.04%	0.01%	0.00%	0.00%
Desalocação de memória	0.00 %	0.00 %	1.11%	1.23%	1.02%	0.93%	0.62%	0.36%
Total do Algoritmo	66.67 %	100.0 0%	99.78 %	99.96 %	100.0 0%	100.0 0%	100.0 0%	100.00 %

- **Tabela 3:** Parâmetros estruturais de cada caso de teste, detalhando o número de vértices n , as dimensões m e os grupos desejados k .

Entrada	n	k	m
1	50	2	2
2	100	4	3
3	1000	5	2
4	2500	5	5
5	5000	10	10
6	10000	20	20
7	20000	40	40
8	40000	80	80

Como observado nos dados colhidos experimentalmente, a análise de complexidade segue o padrão esperado de tempo e de ordem de crescimento. Destaca-se que a maior parte do tempo foi criação das arestas, cuja otimização é especialmente difícil. Contudo, o tempo obtido, mesmo para os casos grandes, é satisfatório.

REFERÊNCIAS

KRUSKAL'S ALGORITHM. **Wikipedia**, 2025. Disponível em: https://en.wikipedia.org/wiki/Kruskal%27s_algorithm. Acesso em: 28 de abril. 2026.

COMARELA, Giovanni. **Trabalho Prático T1**. Vitória, 14 de abril. 2026.